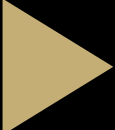




CS-218 Data Structures and Algorithms



LECTURE 20

BY UROOJ AINUDDIN

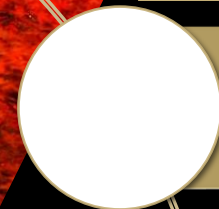
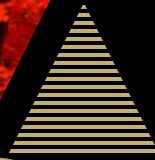




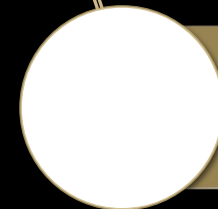
Trees

BOOK 1 CHAPTER 13





We were introduced to the tree structure.



We learned about the terminology associated with trees.



We learned about the binary tree.



We learned to classify binary trees.

In the last lecture...



The binarytree.py file - the class BTree

```
class BTree:
```

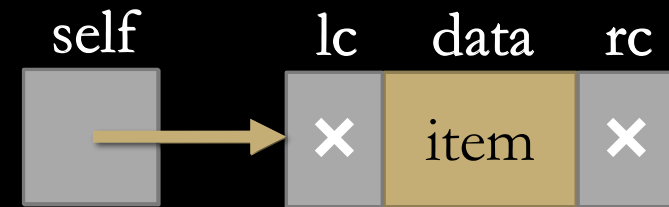
```
    # creates tree root and returns a pointer to it.
```

```
    def __init__(self, item):
```

```
        self.data = item
```

```
        self.lc = None
```

```
        self.rc = None
```



Time: $O(1)$

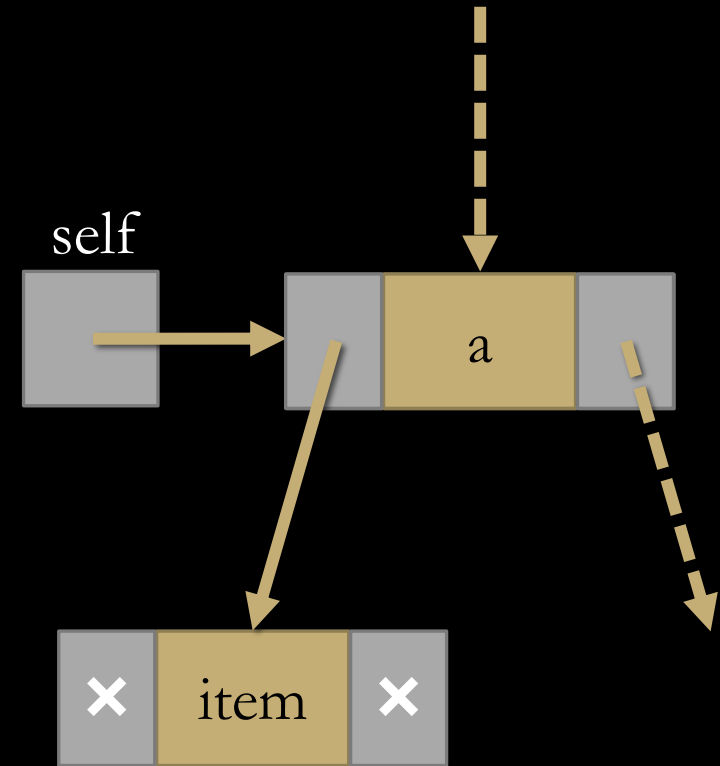


The binarytree.py file - addlc

- The function adds a left child to a node pointed to by self, if the node does not already have a left child.

```
def addlc(self, item):  
    assert self.lc is None, "Left child already present"  
    self.lc = BTree(item)
```

Time: $O(1)$

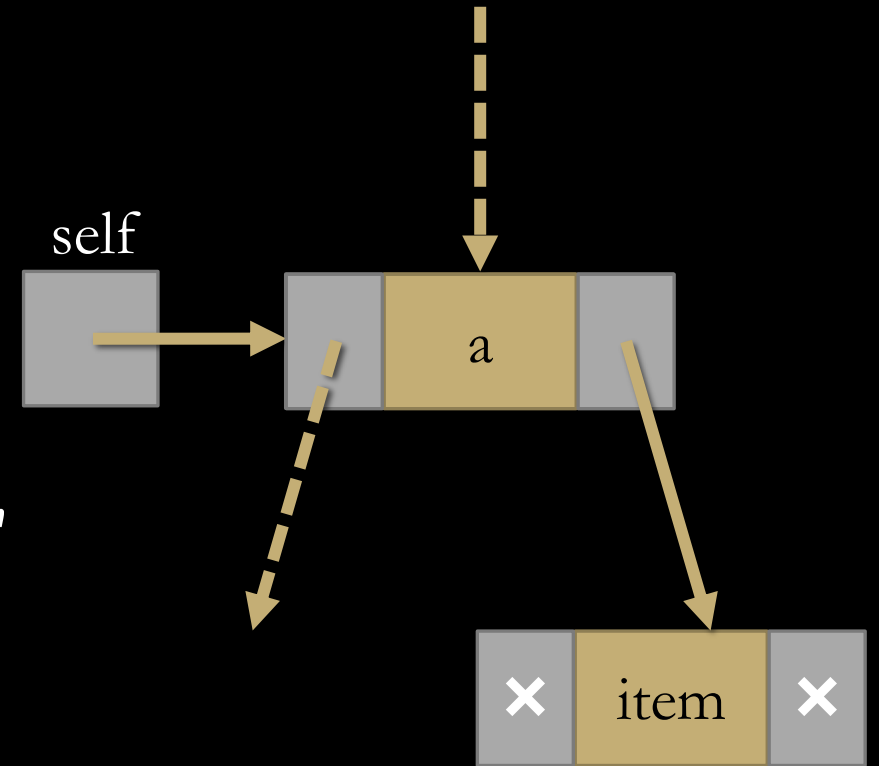


The binarytree.py file - addrc

- The function adds a right child to a node pointed to by self, if the node does not already have a right child.

```
def addrc(self, item):  
    assert self.rc is None, "Right child already present"  
    self.rc = BTree(item)
```

Time: $O(1)$

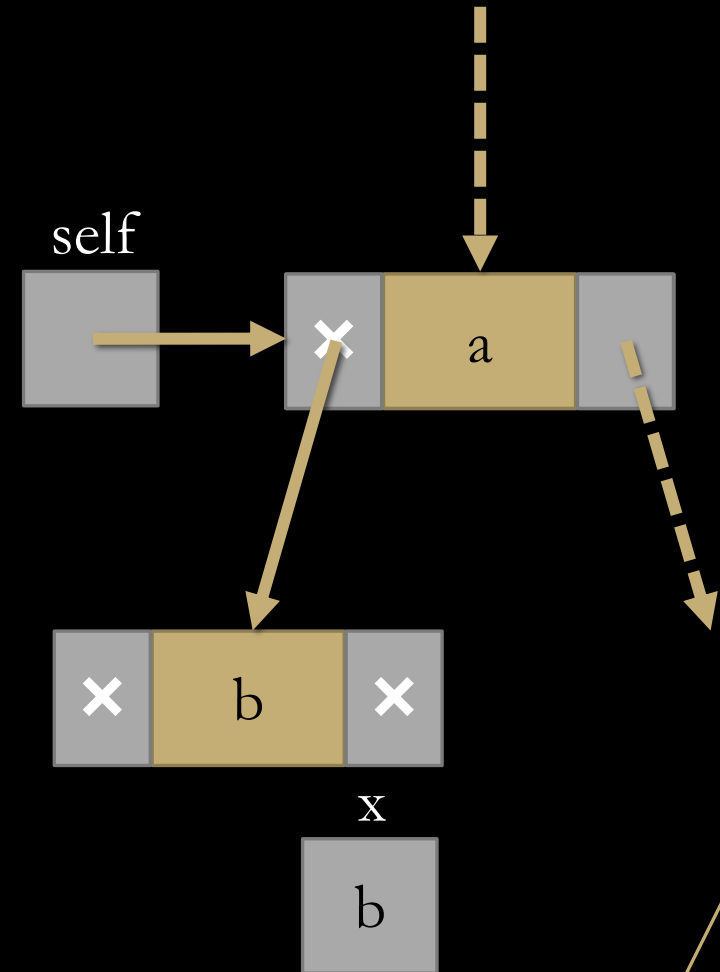


The binarytree.py file - dellc

- The function deletes the left child of a node pointed to by self, if the node has a leaf as its left child. It returns the data of the deleted node.

```
def dellc(self):  
    assert self.lc is not None, "Left child absent"  
    assert self.lc.lc is None and self.lc.rc is None, \  
        "This node has children"  
    x = self.lc.data  
    self.lc = None  
    return x
```

Time: $O(1)$



The binarytree.py file - delrc

- The function deletes the right child of a node pointed to by self, if the node has a leaf as its right child. It returns the data of the deleted node.

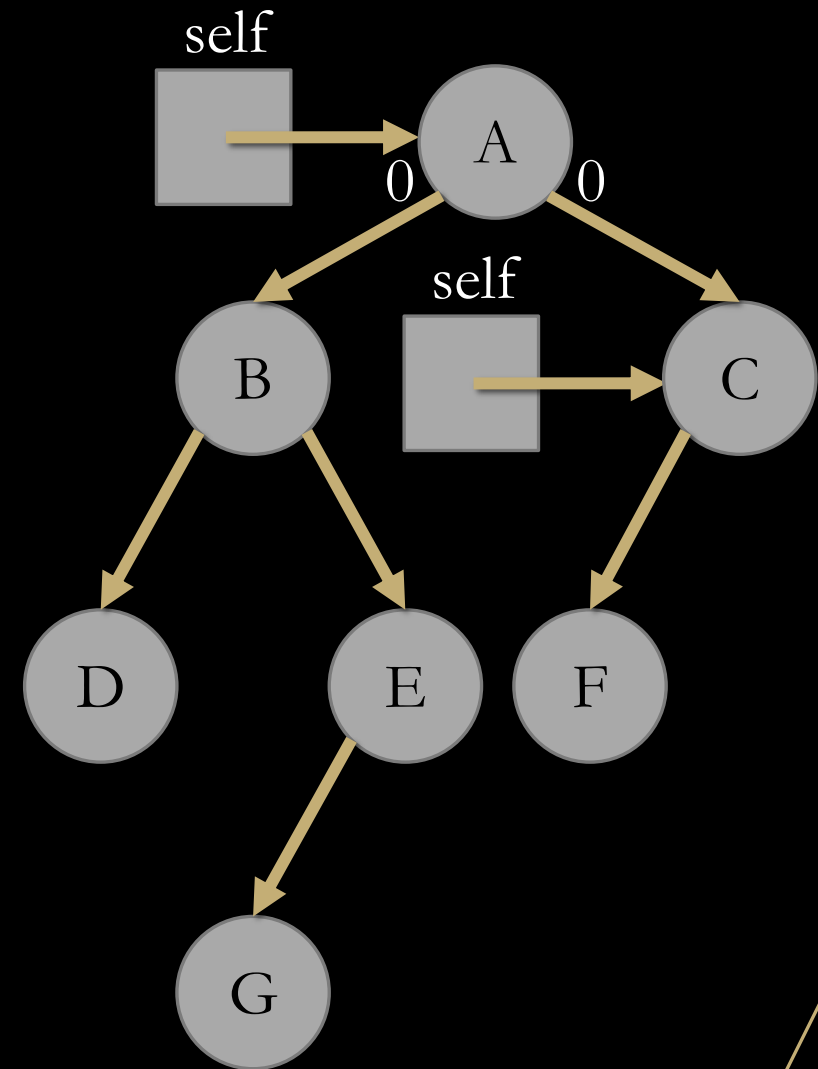
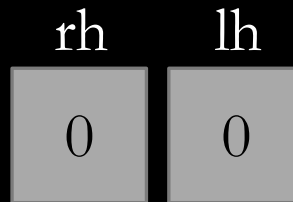
```
def delrc(self):  
    assert self.rc is not None, "Right child absent"  
    assert self.rc.lc is None and self.rc.rc is None,\  
        "This node has children"  
    x = self.rc.data  
    self.rc = None  
    return x
```

Make figures to depict the execution of this code and determine its time complexity.



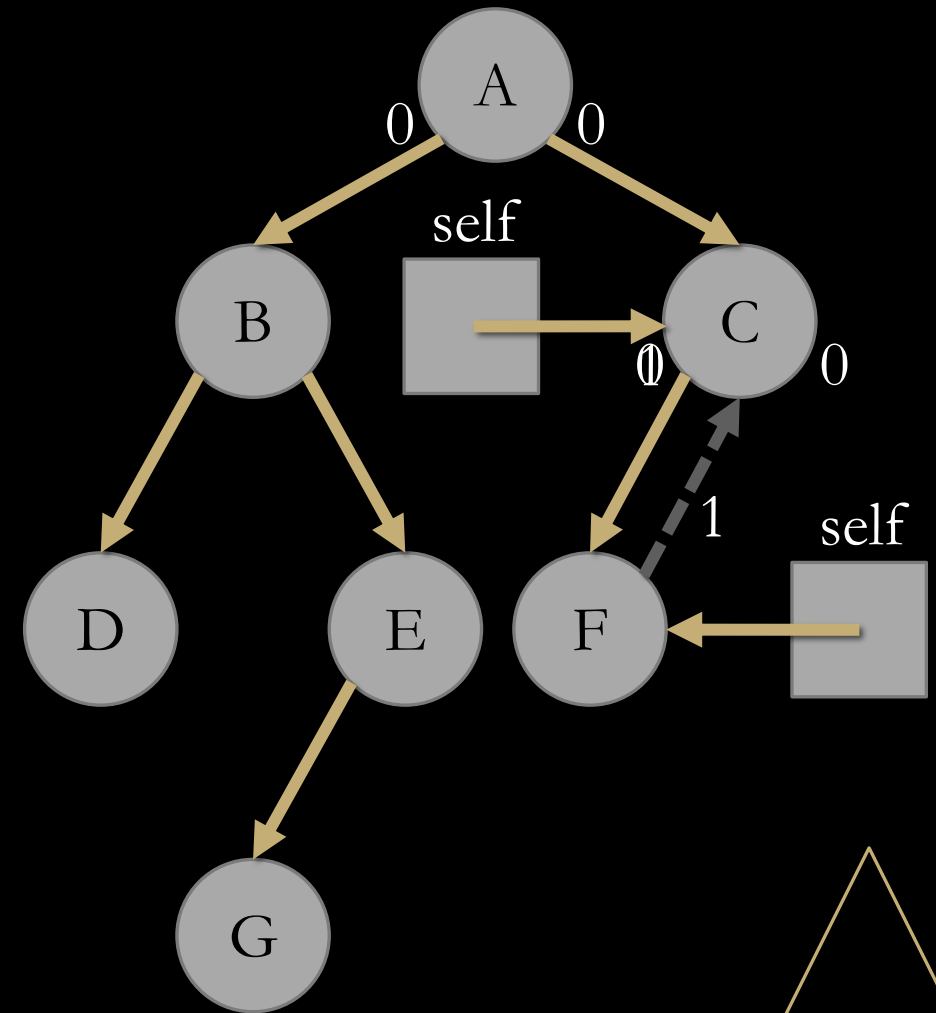
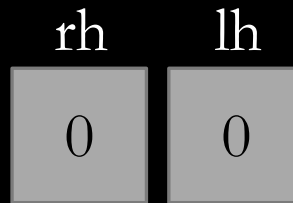
The binarytree.py file - height

```
def height(self):  
    if self.rc is None and self.lc is None:  
        return 1  
    rh = 0  
    lh = 0  
    if self.rc is not None:  
        rh = self.rc.height()  
    if self.lc is not None:  
        lh = self.lc.height()  
    if rh > lh:  
        return rh + 1  
    return lh + 1
```



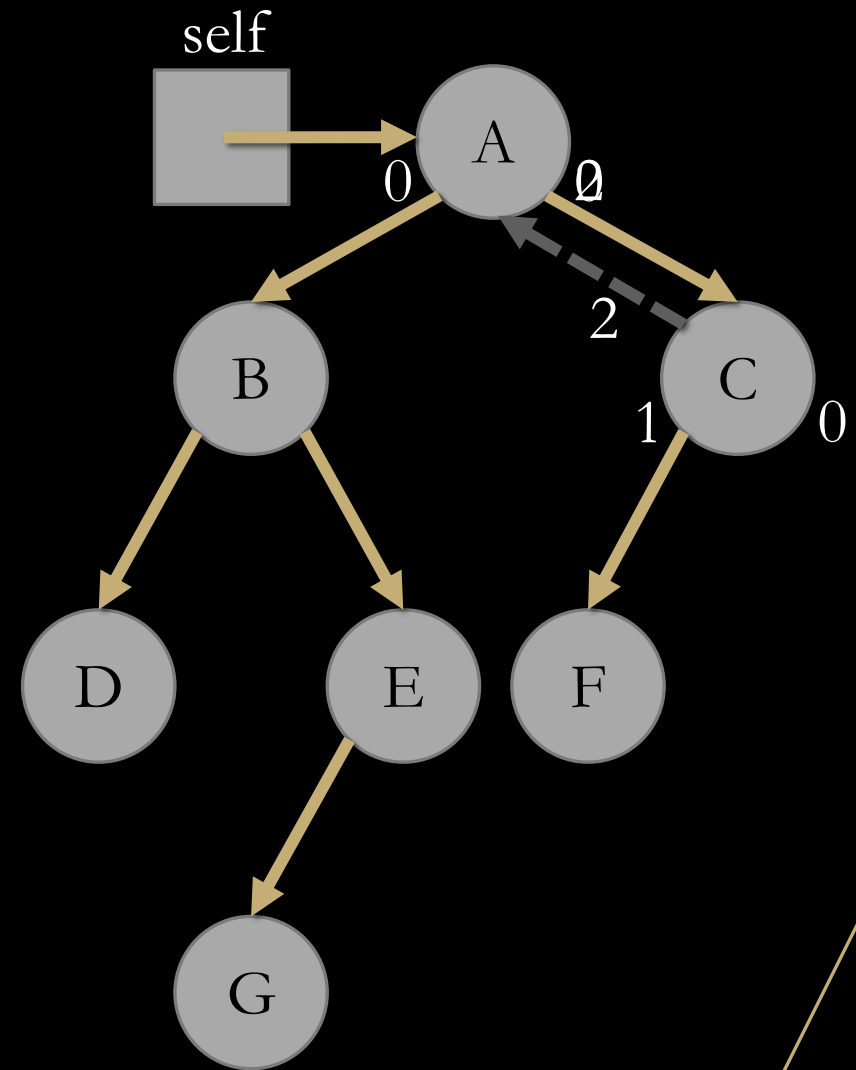
The binarytree.py file - height

```
def height(self):  
    if self.rc is None and self.lc is None:  
        return 1  
    rh = 0  
    lh = 0  
    if self.rc is not None:  
        rh = self.rc.height()  
    if self.lc is not None:  
        lh = self.lc.height()  
    if rh > lh:  
        return rh + 1  
    return lh + 1
```



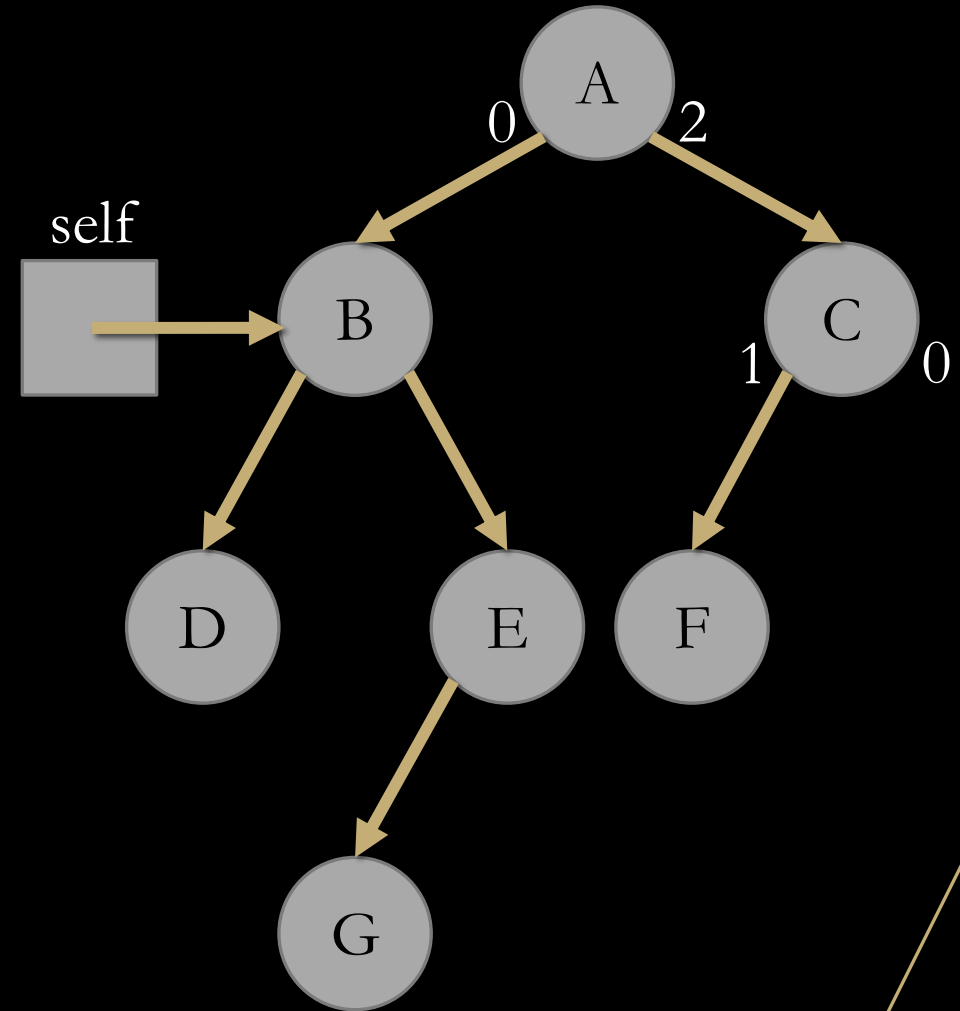
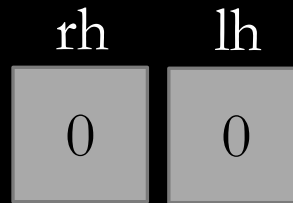
The binarytree.py file - height

```
def height(self):  
    if self.rc is None and self.lc is None:  
        return 1  
    rh = 0  
    lh = 0  
    if self.rc is not None:  
        rh = self.rc.height()  
    if self.lc is not None:  
        lh = self.lc.height()  
    if rh > lh:  
        return rh + 1  
    return lh + 1
```



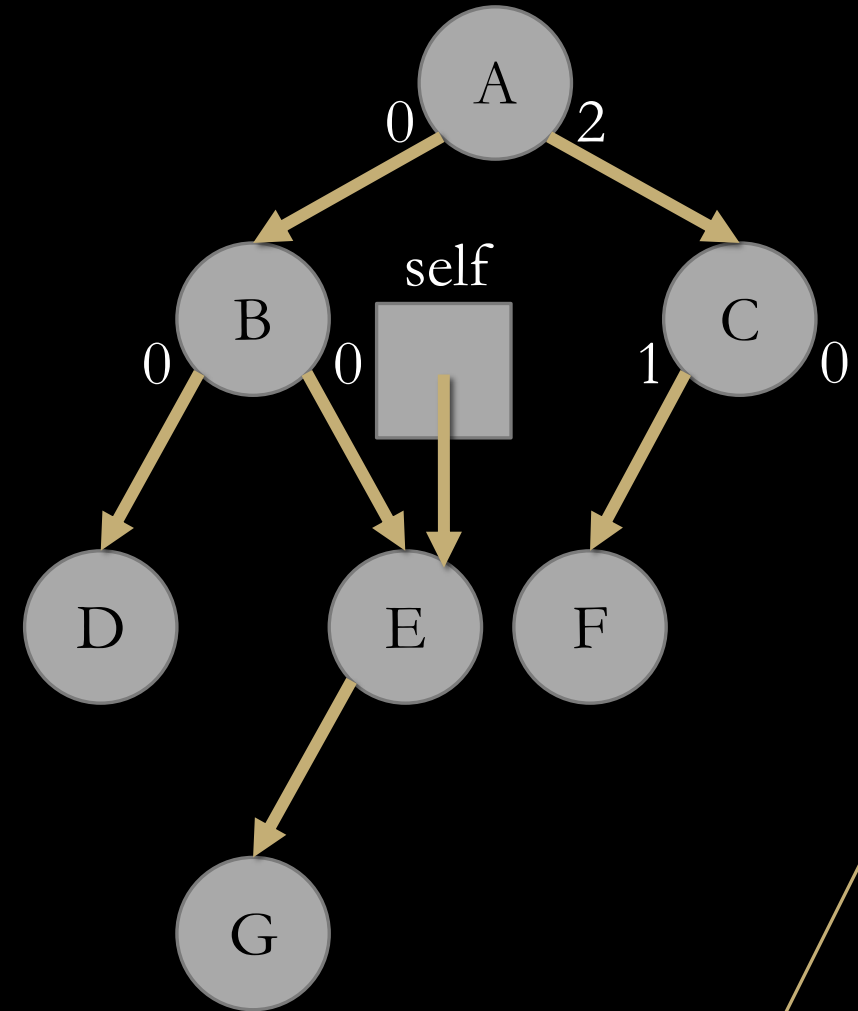
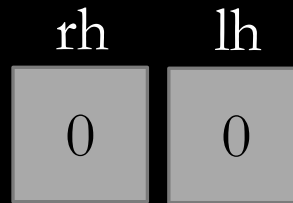
The binarytree.py file - height

```
def height(self):  
    if self.rc is None and self.lc is None:  
        return 1  
    rh = 0  
    lh = 0  
    if self.rc is not None:  
        rh = self.rc.height()  
    if self.lc is not None:  
        lh = self.lc.height()  
    if rh > lh:  
        return rh + 1  
    return lh + 1
```



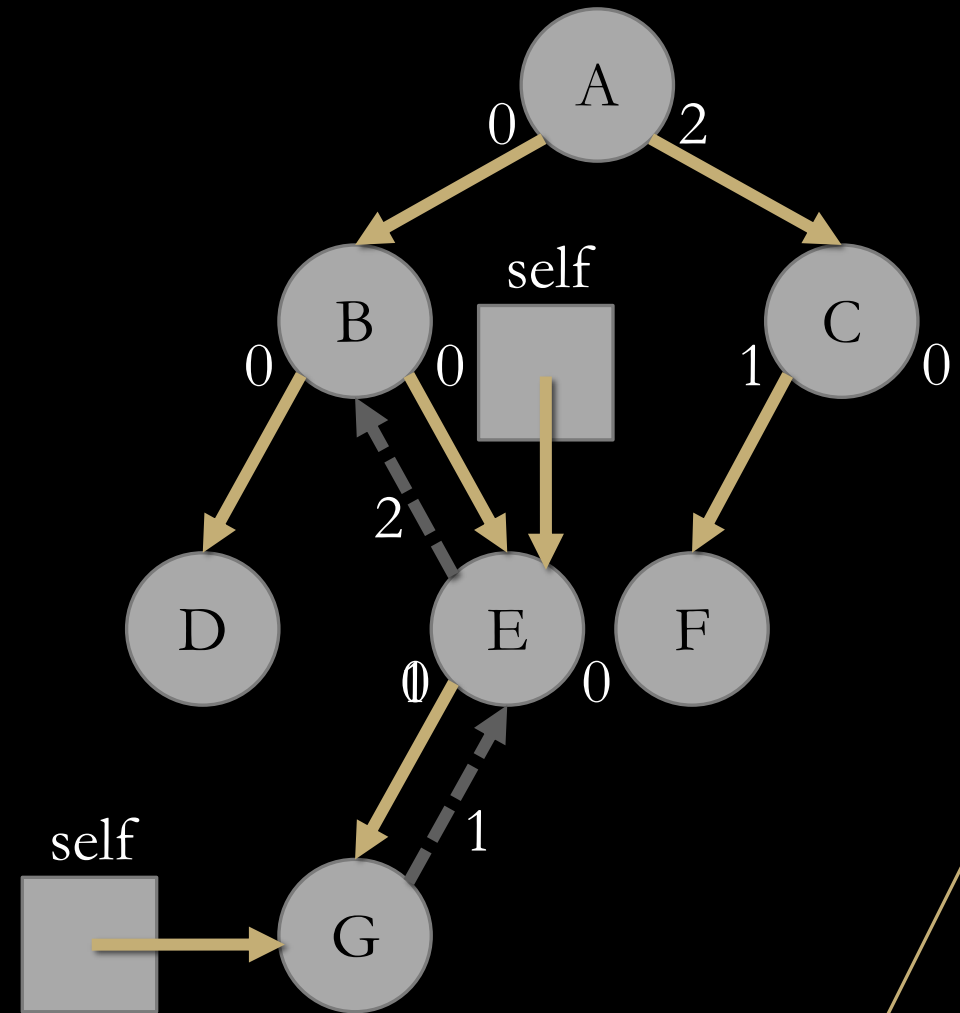
The binarytree.py file - height

```
def height(self):  
    if self.rc is None and self.lc is None:  
        return 1  
    rh = 0  
    lh = 0  
    if self.rc is not None:  
        rh = self.rc.height()  
    if self.lc is not None:  
        lh = self.lc.height()  
    if rh > lh:  
        return rh + 1  
    return lh + 1
```



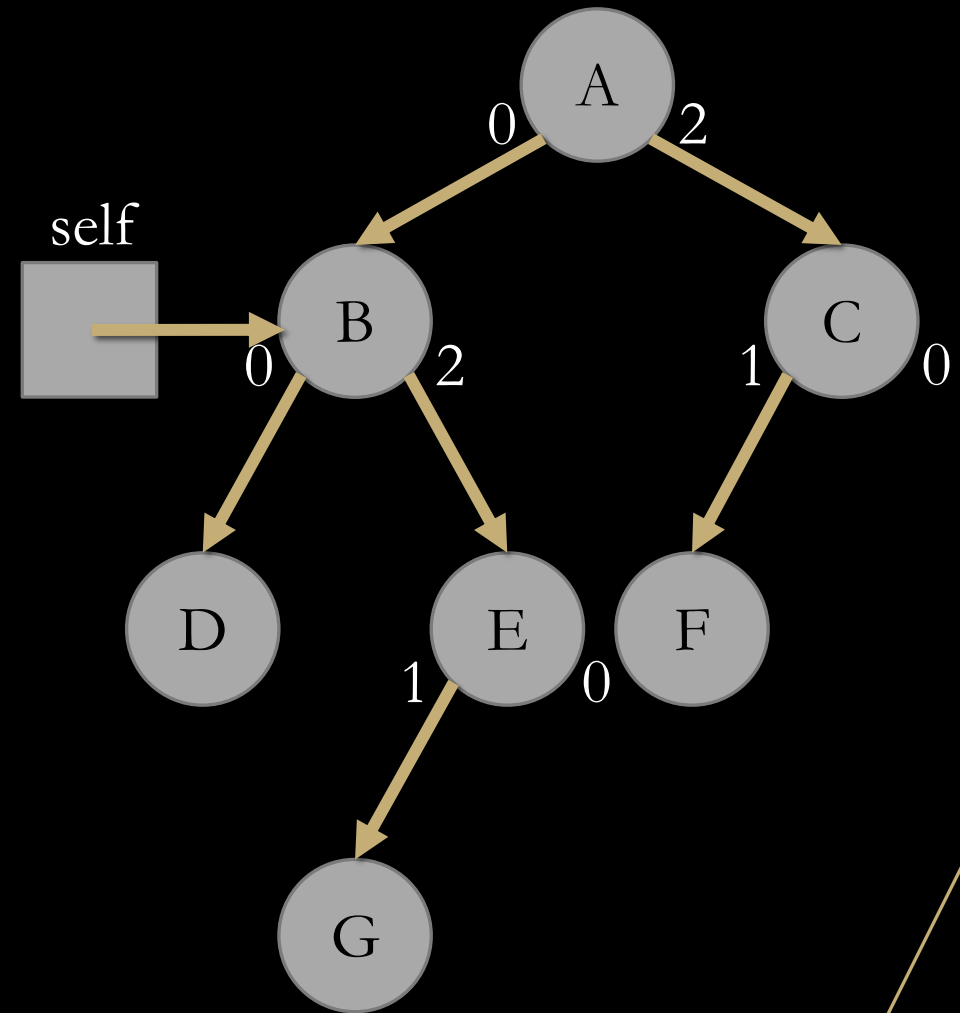
The binarytree.py file - height

```
def height(self):  
    if self.rc is None and self.lc is None:  
        return 1  
    rh = 0  
    lh = 0  
    if self.rc is not None:  
        rh = self.rc.height()  
    if self.lc is not None:  
        lh = self.lc.height()  
    if rh > lh:  
        return rh + 1  
    return lh + 1
```



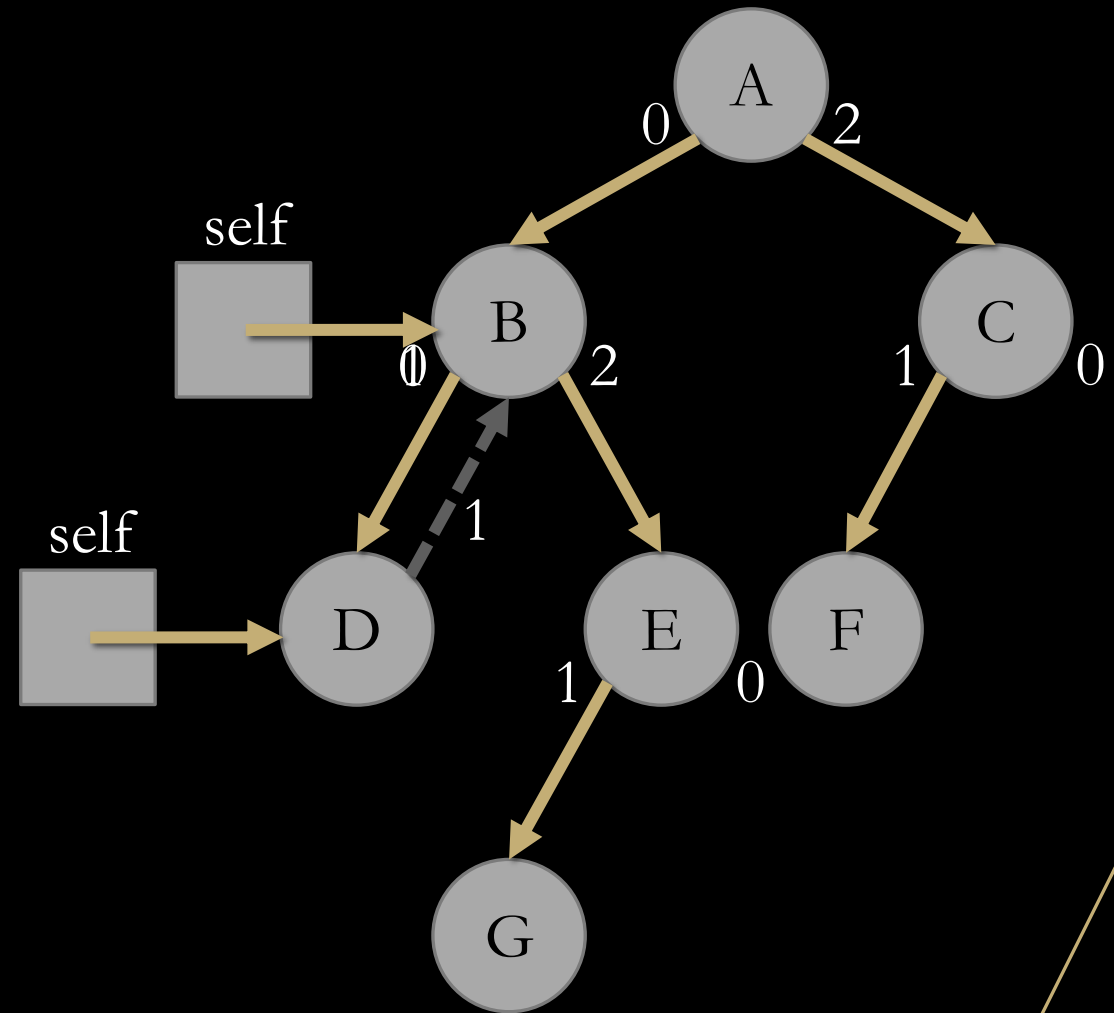
The binarytree.py file - height

```
def height(self):  
    if self.rc is None and self.lc is None:  
        return 1  
    rh = 0  
    lh = 0  
    if self.rc is not None:  
        rh = self.rc.height()  
    if self.lc is not None:  
        lh = self.lc.height()  
    if rh > lh:  
        return rh + 1  
    return lh + 1
```



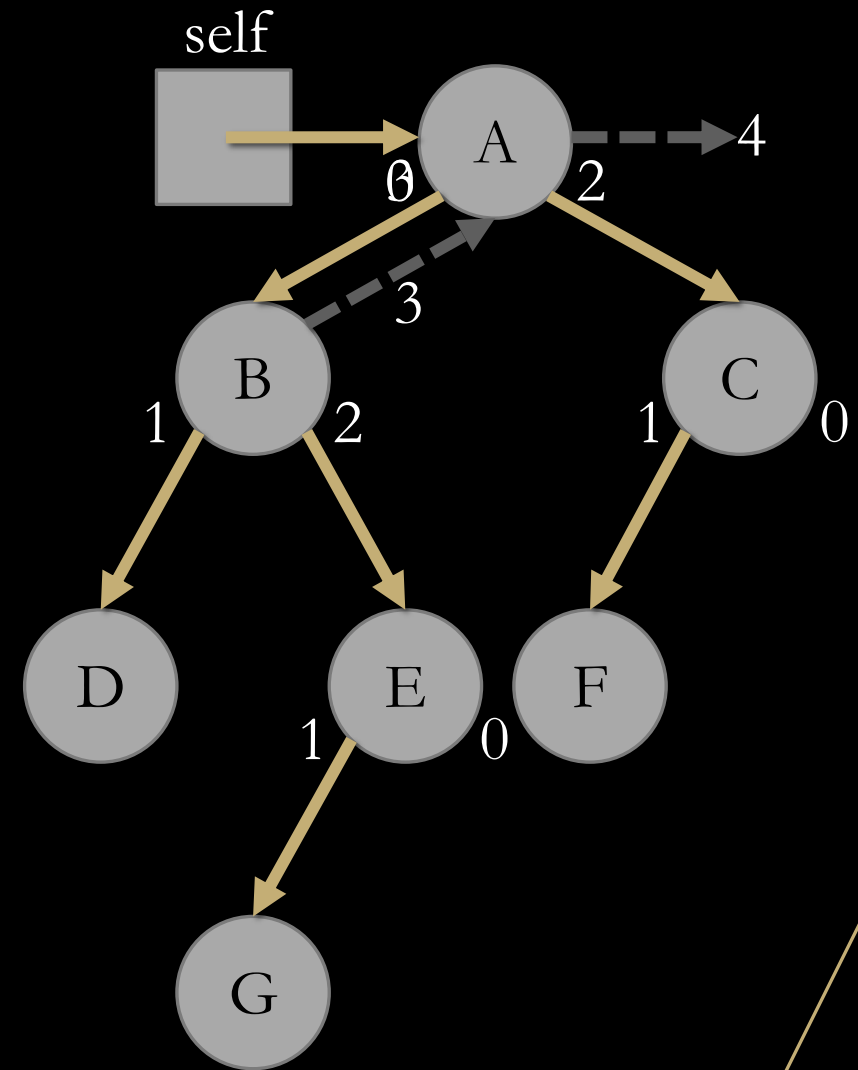
The binarytree.py file - height

```
def height(self):  
    if self.rc is None and self.lc is None:  
        return 1  
    rh = 0  
    lh = 0  
    if self.rc is not None:  
        rh = self.rc.height()  
    if self.lc is not None:  
        lh = self.lc.height()  
    if rh > lh:  
        return rh + 1  
    return lh + 1
```



The binarytree.py file - height

```
def height(self):  
    if self.rc is None and self.lc is None:  
        return 1  
    rh = 0  
    lh = 0  
    if self.rc is not None:  
        rh = self.rc.height()  
    if self.lc is not None:  
        lh = self.lc.height()  
    if rh > lh:  
        return rh + 1  
    return lh + 1
```



The binarytree.py file - height

```
def height(self):
    if self.rc is None and self.lc is None:
        return 1
    rh = 0
    lh = 0
    if self.rc is not None:
        rh = self.rc.height()
    if self.lc is not None:
        lh = self.lc.height()
    if rh > lh:
        return rh + 1
    return lh + 1
```

- The function is called for every node.
- The base case of recursion materializes at the leaves, which return 1.
- Consider the number of nodes in the tree to be n .
- A single function instance takes $O(1)$ time.

Time: $O(n)$



The binarytree.py file - height

```
def height(self):
    if self.rc is None and self.lc is None:
        return 1
    rh = 0
    lh = 0
    if self.rc is not None:
        rh = self.rc.height()
    if self.lc is not None:
        lh = self.lc.height()
    if rh > lh:
        return rh + 1
    return lh + 1
```

Space: $O(h)$

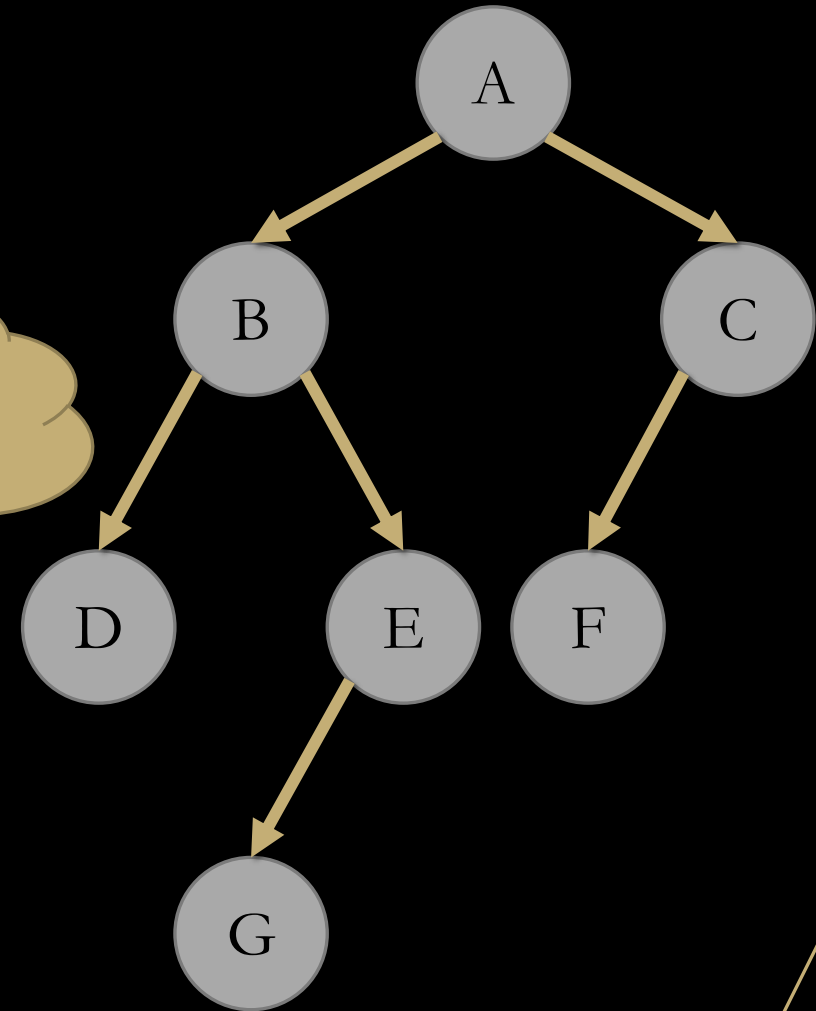
- The function uses 2 variables and the run-time stack.
- The maximum number of activation records are pushed in the longest branch of the tree.
- The base case of recursion materializes at the leaf of the longest branch, which returns 1.
- The number of activation records pushed in the longest branch of the tree with height h is $h - 1$.
- A single function instance takes $O(1)$ space.



The binarytree.py file - nodecount

```
def nodecount(self):  
    if self.lc is None and self.rc is None:  
        return 1  
    lnc = 0  
    rnc = 0  
    if self.lc is not None:  
        lnc = self.lc.nodecount()  
    if self.rc is not None:  
        rnc = self.rc.nodecount()  
    return lnc + rnc + 1
```

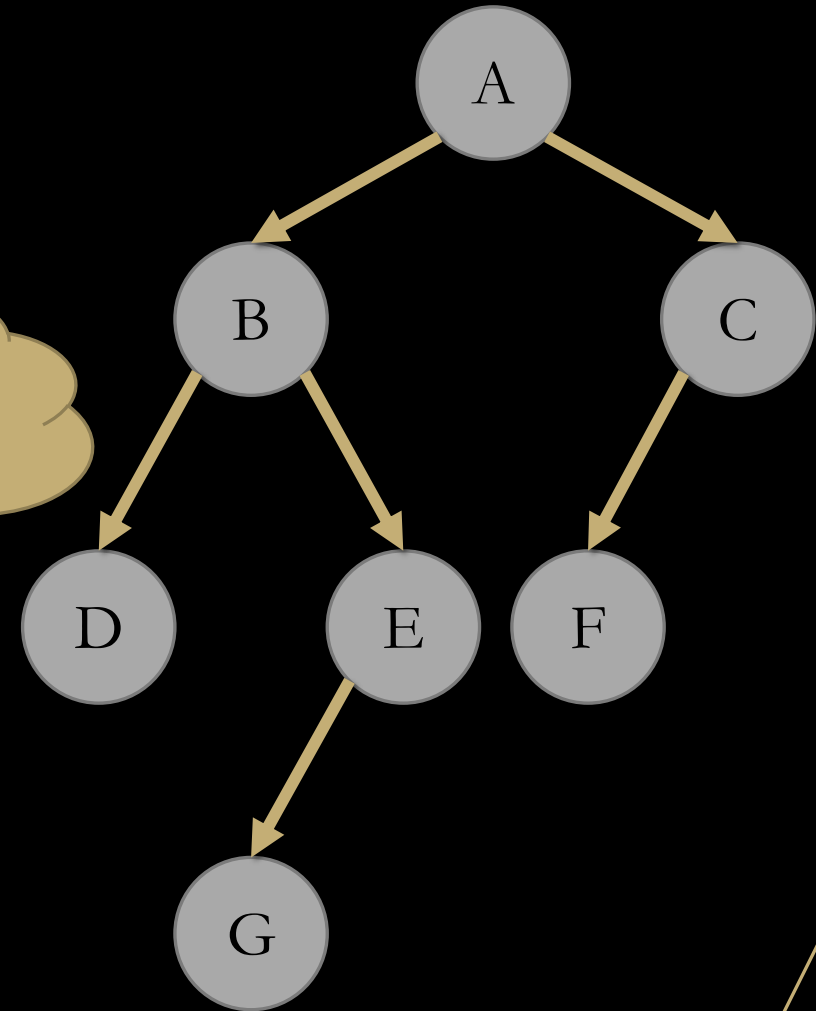
Run the code for the tree drawn here and find the time and space complexity.



The binarytree.py file - leafcount

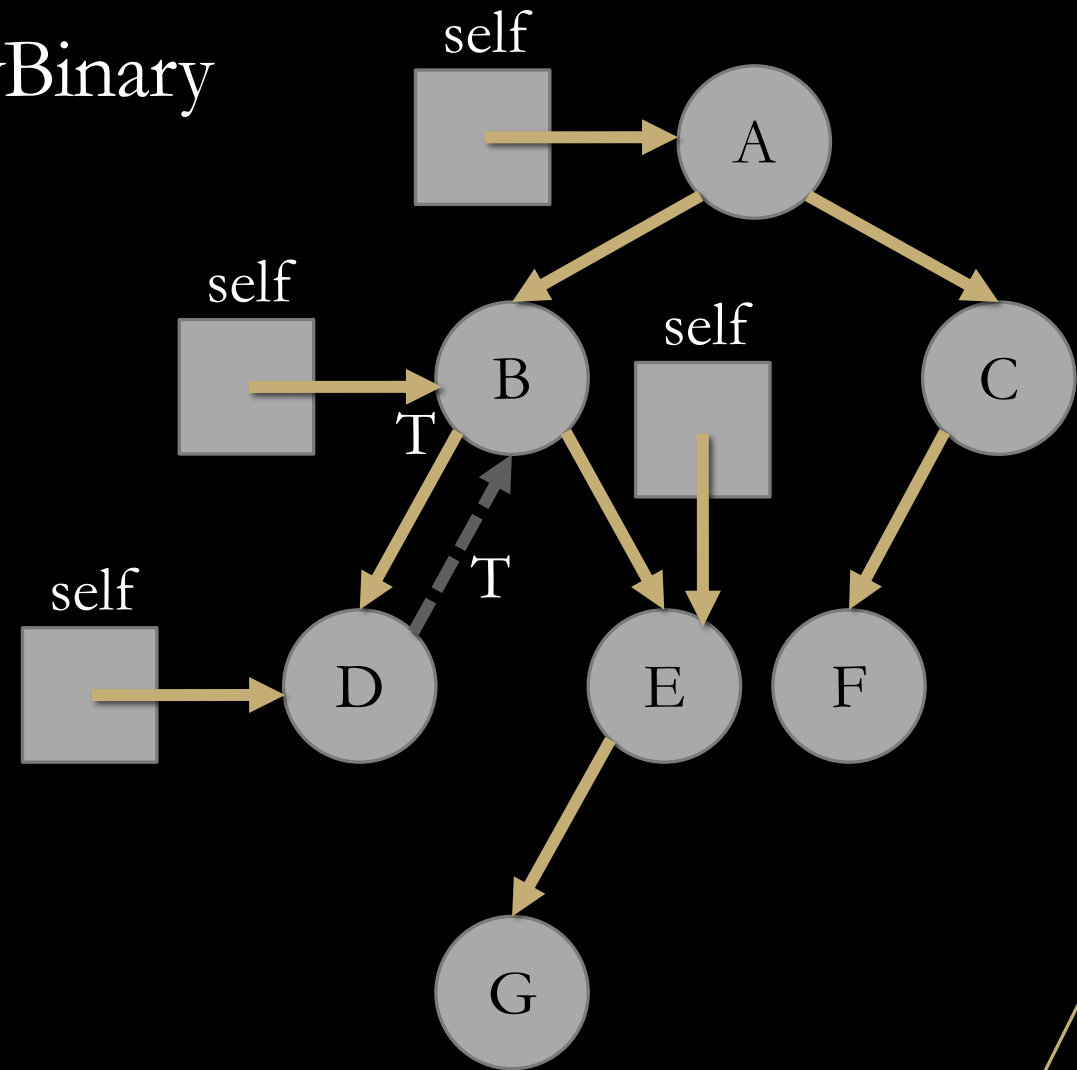
```
def leafcount(self):  
    if self.lc is None and self.rc is None:  
        return 1  
    llc = 0  
    rlc = 0  
    if self.lc is not None:  
        llc = self.lc.leafcount()  
    if self.rc is not None:  
        rlc = self.rc.leafcount()  
    return llc + rlc
```

Run the code for the tree drawn here and find the time and space complexity.



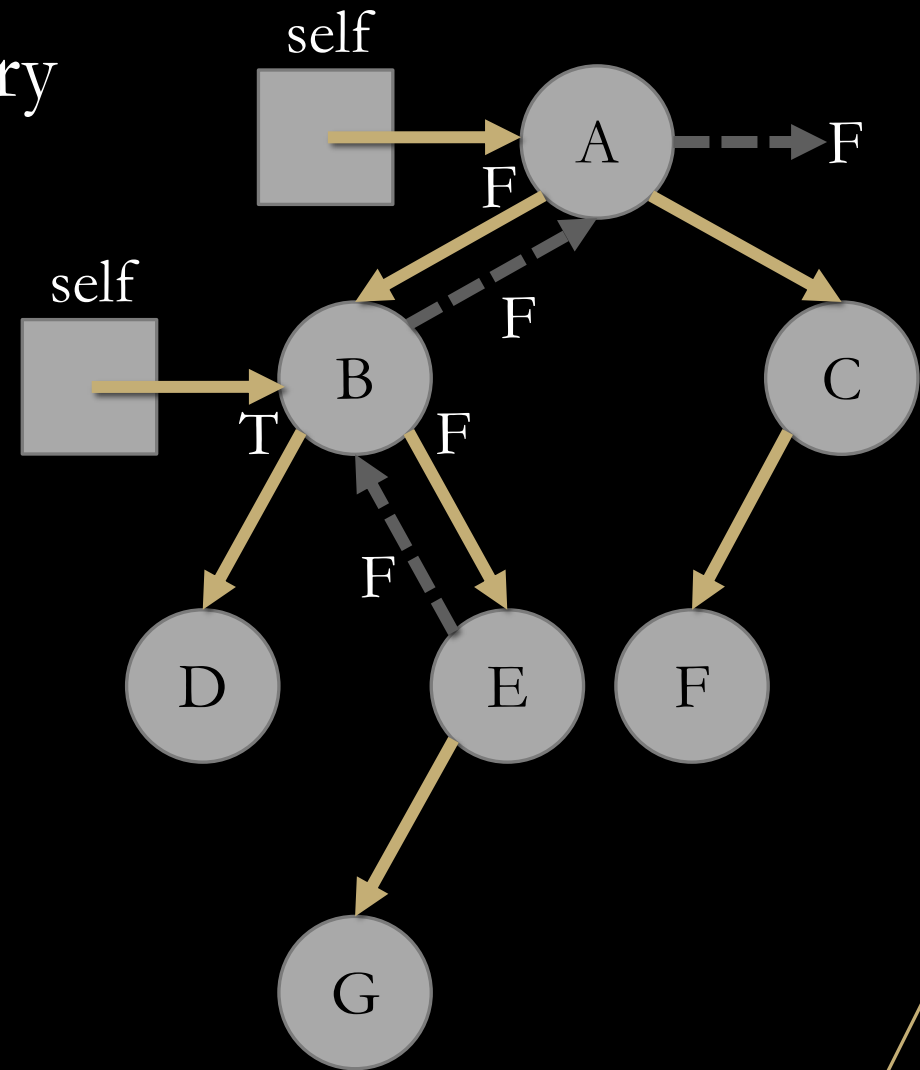
The binarytree.py file - isStrictlyBinary

```
def isStrictlyBinary(self):  
    if self.lc is None:  
        if self.rc is None:  
            return True  
        return False  
    if self.rc is None:  
        return False  
    return self.lc.isStrictlyBinary() \  
        and self.rc.isStrictlyBinary()
```



The binarytree.py file - isStrictlyBinary

```
def isStrictlyBinary(self):  
    if self.lc is None:  
        if self.rc is None:  
            return True  
        return False  
    if self.rc is None:  
        return False  
    return self.lc.isStrictlyBinary() \  
        and self.rc.isStrictlyBinary()
```



The binarytree.py file - isStrictlyBinary

```
def isStrictlyBinary(self):  
    if self.lc is None:  
        if self.rc is None:  
            return True  
        return False  
    if self.rc is None:  
        return False  
    return self.lc.isStrictlyBinary() \  
        and self.rc.isStrictlyBinary()
```

Time: $O(2^h)$

- Worst case: The maximum number of function instances are called when the tree is a PBT.
- The function is called for every node.
- The base case of recursion materializes at the leaves in the last level, which return True.
- The number of nodes in a PBT of height h is $2^h - 1$.
- A single function instance takes $O(1)$ time.



The binarytree.py file - isStrictlyBinary

```
def isStrictlyBinary(self):  
    if self.lc is None:  
        if self.rc is None:  
            return True  
        return False  
    if self.rc is None:  
        return False  
    return self.lc.isStrictlyBinary() \  
        and self.rc.isStrictlyBinary()
```

Space: $O(h)$

- The maximum number of activation records are pushed in the longest branch of the tree.
- The base case of recursion materializes at the leaf of the longest branch, which returns True.
- The number of activation records pushed in the longest branch of the tree with height h is $h - 1$.
- A single function instance takes $O(1)$ space.



The binarytree.py file - isPerfect

```
def isPerfect(self):  
    h = self.height()  
    n = self.nodecount()  
    m = pow(2, h) - 1  
    if n == m:  
        return True  
    return False
```

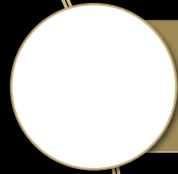
Find the time and
space complexity.

Write an equivalent
recursive function.

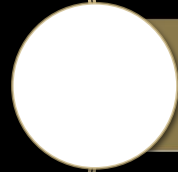




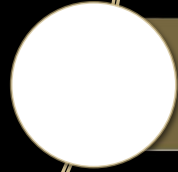
We implemented a class for the binary tree data structure.



We implemented functions to add and remove nodes from a binary tree.



We implemented codes to calculate height, and count nodes and leaves of a binary tree.



We implemented codes to determine the type of a binary tree.



We examined all codes for time and space complexities.

So, what did we learn today?



Things to do



 Read the book!

 Email suggestions on content or quality of this lecture at uroojain@neduet.edu.pk

 Note your questions and put them up in the relevant online session.